QuickHire Technical Implementation Document (TID)

Project: QuickHire - AI-Powered Hiring Workflow Platform
Version: 1.0
Date: 2026-03-04
Primary Codebase: /Users/mik/Desktop/code/QuickHire

---

1. Introduction

1.1 Purpose of This Document

This Technical Implementation Document (TID) provides a full engineering-level description of QuickHire's implementation. It is written for technical readers who need to understand architecture, runtime behavior, AI integration, module boundaries, key functions, deployment strategy, and operational constraints.

The document is intended to support:
- onboarding of new engineers,
- technical review and audit,
- deployment and operations handover,
- extension planning and refactoring decisions.

1.2 Project Overview

QuickHire is a web application for recruiter and hiring-team workflows. It ingests job descriptions and candidate resumes, uses an LLM to score candidate-job fit, and provides operational capabilities including:
- job posting lifecycle management,
- resume upload and extraction,
- AI-based candidate scoring and ranking,
- candidate reporting and onboarding document generation,
- communication workflows (invites, decisions, custom emails),
- analytics dashboards and CSV export,
- account settings and data management controls.

1.3 Problem Statement

Manual resume screening is slow, inconsistent, and difficult to scale when candidate volume is high. QuickHire solves this by:
- normalizing resume and JD data into structured candidate records,
- applying consistent scoring logic via AI plus deterministic post-processing,
- providing workflow controls for recruiter actions and hiring outcomes,
- exposing a full operational dashboard for decision execution.

---

2. Key Components of the Program

2.1 Architecture Overview

2.1.1 System Design

QuickHire uses a layered Flask monolith with modular boundaries:

- Presentation layer: Jinja templates + Vanilla JavaScript (static/js/*).

- API/Controller layer: Flask blueprints (routes/*, routes/api/*).
- Service layer: integrations and domain services (services/*).
- Persistence layer: SQLAlchemy ORM models (user_model.py) backed by PostgreSQL.
- Utility layer: formatting, parsing, analytics shaping (utils/formatting.py).

It is deployed as a serverless Flask app on Vercel via rewrite to api/index.py.

2.1.2 High-Level Request Flow

```
Browser (Jinja + JS)
   -> Flask Blueprint Route (/dashboard/...)
      -> Service Layer (AI, storage, email, PDF)
         -> PostgreSQL (SQLAlchemy)
         -> Anthropic API (Claude)
         -> Supabase Storage (optional on Vercel)
         -> Gmail SMTP
   <- JSON or HTML response
```

2.2 Technologies Used

```
| Layer | Technology | Why It Is Used |
|---|---|---|
| Web framework | Flask 3.1.2 | Lightweight, explicit routing, easy blueprint modularity |
| Auth/session | Flask-Login 0.6.3 | Session auth with login-required route guards |
| ORM | Flask-SQLAlchemy 3.1.1 + SQLAlchemy 2.0.46 | Strong ORM typing and relational modeling
|
| Database | PostgreSQL (psycopg2-binary) | Durable relational data for users/jobs/candidates
|
| LLM integration | Anthropic SDK 0.84.0 | JD analysis, candidate scoring, onboarding
extraction |
| PDF text extraction | pdfplumber 0.11.9 | Resume/JD text extraction from PDFs |
| PDF generation | fpdf2 2.8.6 | Candidate report and onboarding document output |
| Cloud object storage | Supabase Python SDK 2.28.0 | Optional document storage and cleanup |
| Email transport | Gmail SMTP over SSL | Invitation and decision messaging |
| Deployment | Vercel serverless (vercel.json) | Managed deployment with rewrite routing |
| Frontend runtime | Vanilla JS + Jinja templates | Lower complexity and fast iteration
without SPA overhead |
```

2.3 Core Modules and Key Functions

2.3.1 Application Bootstrap

File: /Users/mik/Desktop/code/QuickHire/main.py

Responsibilities:
- loads env configuration,
- validates required env vars (SECRET_KEY, DATABASE_URL),
- initializes Flask, DB, login manager,
- registers all blueprints,
- applies pgbouncer-safe DB connection behavior,
- sets security headers,
- registers custom error pages.

Key snippet

```
_required_env = ["SECRET_KEY", "DATABASE_URL"]
_missing = [v for v in _required_env if not os.getenv(v)]
if _missing:
    sys.exit(f"FATAL: Missing required environment variables: {', '.join(_missing)}")
```

2.3.2 Domain Models

File: /Users/mik/Desktop/code/QuickHire/user_model.py

Core entities:
- User: account, preferences, notification flags.
- Job: JD content, metadata, status lifecycle.
- Candidate: extracted resume text, scores, outcomes.
- ResetToken: password reset flow with expiry.

Design rationale:
- strict one-to-many: User -> Jobs -> Candidates,
- status fields for workflow-state transitions,
- JSON-as-text fields (matched_skills, required_skills) for flexible schema evolution.

2.3.3 Job Ingestion and JD Analysis

File: /Users/mik/Desktop/code/QuickHire/routes/api/jobs.py

Function: upload_jd()
- Input: multipart form with jd_file or plain jd_text.
- Logic: validate PDF extension, extract text, optional storage upload, infer title/department/location, create Job row.
- Output: JSON with job_id and inferred title.

Function: analyze_jd(job_id)
- Input: job id.
- Logic: calls Claude with strict JSON schema prompt for role metadata extraction.
- Output: structured analysis payload for frontend rendering.

Key snippet

```
system_msg, user_msgs = build_jd_analysis_prompt(job.jd_text)
raw = call_claude(client, user_msgs, system=system_msg)
result = parse_ai_json(raw)
```

2.3.4 Resume Upload and Candidate Creation

File: /Users/mik/Desktop/code/QuickHire/routes/api/candidates.py

Function: upload_resumes(job_id)
- Input: multipart resumes files.
- Logic:
- validates ownership and file presence,
- enforces PDF-only handling,
- extracts text with serverless limits (max_pages, max_chars),
- optional storage upload,
- creates candidate rows,
- robust rollback and explicit JSON error mapping.
- Output: JSON candidates array (id, filename) or explicit failure reason.

Current serverless guardrails
- default parse limits on Vercel: RESUME_PARSE_MAX_PAGES=3, RESUME_PARSE_MAX_CHARS=20000 (implicit defaults if unset).

Key snippet

```
max_pages, max_chars = _resume_parse_limits()
resume_text = extract_pdf_text(file_bytes, max_pages=max_pages, max_chars=max_chars)
```

2.3.5 AI Scoring Pipeline

Files:
- /Users/mik/Desktop/code/QuickHire/routes/api/candidates.py
- /Users/mik/Desktop/code/QuickHire/services/ai.py

Model used
- claude-sonnet-4-20250514 via Anthropic Messages API.

Function chain
1. route obtains pending candidates for a job,
2. route processes candidates in batches (SCREENING_BATCH_SIZE, default 1 on Vercel),
3. service builds a structured scoring prompt,
4. service calls Claude with retry policy on rate limits,
5. service parses JSON payload and clamps scores to [0, 100],
6. route stores results and returns ranked response.

Key snippet

```
kwargs = {
    "model": "claude-sonnet-4-20250514",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": prompt}],
}
```

2.3.6 Candidate Workflow Actions

File: /Users/mik/Desktop/code/QuickHire/routes/api/candidates.py

Major actions:
- send_invites: marks status invited, sends SMTP invite email.
- final_decision: sets final_hired or final_rejected, sends decision email.
- send_custom_email: direct recruiter message path.
- generate_onboarding: generates onboarding PDF (AI-assisted extraction optional).

Parameters and returns (example)
- POST /dashboard/send-invites
- input: candidate_ids[], scheduling_link, message
- output: { success, results: [{ id, email_sent, name }] }

2.3.7 Analytics Computation

File: /Users/mik/Desktop/code/QuickHire/utils/formatting.py

Function: compute_analytics_data(user_id, range_param)
- aggregates KPI, over-time counts, funnel conversion, dept distribution, top skills, recent hires,
- outputs frontend-ready JSON structure,
- handles empty datasets safely.

Data structures used
- defaultdict(int) for counter accumulation,
- lists of dicts for chart payloads,
- deterministic sorting by count and date.

2.3.8 Frontend Modules

```
| File | Responsibility |
|---|---|
```

```
| static/js/dashboard.js | 4-step recruiter wizard, upload, screening, result actions |
| static/js/jobs.js | job list/detail view, candidate upload/analyze/invite from job detail |
| static/js/candidates.js | candidates page interactions |
| static/js/analytics.js | dashboard analytics visual rendering |
| static/js/settings.js | settings persistence and destructive action confirmations |
```

2.3.9 Frontend Component Details (Brief)

Key UI components are implemented as server-rendered template sections with JS state orchestration:

- Dashboard wizard (Step 1-4):
- Step 1: JD upload/paste and optional AI JD analysis.
- Step 2: Resume upload (sequential per-file requests).
- Step 3: Screening progress UI with batched polling.
- Step 4: Candidate status management (invited/hired/rejected).
- Job detail candidates panel:
- inline upload, analyze, resume preview, invite, and delete actions.
- candidate cards rendered with score badges and status chips.
- Shared modal components:
- resume preview modal (iframe),
- invite modal,
- final decision modal,
- destructive-action confirmations in settings.
- Analytics view components:
- KPI cards,
- SVG bar chart for applications over time,
- funnel rows, top-skills pills, and recent hires table.

This is a progressive-enhancement model: critical pages render from Jinja first, then JS adds dynamic behavior.

2.3.10 CSS System Details (Brief)

Global styling is centralized in static/css/styles.css and follows a tokenized design system:

- Theme tokens (CSS variables):
- color scale (--color-canvas, --color-surface, --color-primary, --color-danger, etc.),
- spacing scale (--spacing-xs to --spacing-3xl),
- typography scale and family (Inter stack),
- motion and shadow tokens (--transition-*, --shadow-*).
- Core UI primitives:
- cards, buttons, badges, pills, form controls, table wrappers.
- Workflow-specific styles:
- wizard progress UI, result cards, score bars, status chips, and dashboard panels.
- Modal system:
- shared .modal and .modal-overlay patterns with reusable header/body/footer blocks.
- Responsive behavior:
- media-query adaptations for navigation, layout spacing, and panel density.

The token system makes color/spacing/typography changes low-risk and consistent across templates.

2.3.11 Important Helper Functions

Several helper functions are critical to reliability and output consistency:

- extract_pdf_text(source, max_pages=None, max_chars=None) (utils/formatting.py)
- parses PDF bytes/streams into text,

- supports hard limits to keep serverless requests bounded.
- format_jd_text(text) (utils/formatting.py)
- transforms raw JD text into structured HTML blocks (headings, lists, paragraphs) for safe rendering.
- serialize_candidate(candidate) (utils/formatting.py)
- normalizes ORM candidate objects into API-safe JSON with defaults and decoded matched_skills.
- parse_ai_json(raw_text) (services/ai.py)
- strips markdown fences and extracts JSON object payloads from model output.
- clamp_score(value) (services/ai.py)
- enforces integer score bounds [0, 100] regardless of model output drift.
- _screening_batch_size() (routes/api/candidates.py)
- central batch-size control for timeout management (1 by default on Vercel).
- _resume_parse_limits() (routes/api/candidates.py)
- centralizes page/char extraction limits with env overrides.

These helpers reduce duplicated logic in routes and are key to stability under mixed data quality and serverless constraints.

---

3. Implementation Strategy

3.1 Development Process

The implementation follows an iterative, feature-slice workflow aligned with Agile principles:
- deliver vertical features (route + service + UI) per increment,
- validate behavior in deployed serverless context,
- harden based on runtime signals (timeouts, storage latency, API rate limits),
- refactor for resiliency without major architecture rewrites.

Evidence of this approach is visible in incremental commits around:
- serverless timeout mitigation,
- sequential upload execution,
- batched screening,
- improved error surfacing,
- invite-flow consistency fixes.

3.2 Key Programming Concepts

3.2.1 Layered Separation of Concerns

- routes handle HTTP and auth ownership checks,
- services encapsulate external integration logic,
- models define state and relationships,
- utils perform formatting and analytics transforms,
- JS modules own page-specific UI behavior.

This reduces coupling and keeps side effects localized.

3.2.2 Defensive Parsing and Validation

- strict PDF extension checks,
- secure_filename for file names,
- JSON parsing hardening for AI outputs,
- score clamping and string truncation,
- explicit ownership checks (candidate.job.user_id == current_user.id).

3.2.3 Design Patterns Used

- Facade-like services: AI, storage, email wrappers isolate vendor API details.
- Controller-Service split: route delegates heavy logic to service/util modules.
- State machine by status fields: job and candidate states drive flow gating.
- Progressive enhancement in frontend: baseline server-rendered pages with JS-driven interaction.

3.3 AI Algorithm Design and Rationale

QuickHire intentionally uses LLM + deterministic post-processing instead of pure ML model hosting.

Why:
- lower infra complexity (no dedicated model serving stack),
- better zero-shot handling for varied resume formats,
- faster schema changes through prompt updates.

Deterministic controls ensure stability:
- strict JSON schema prompt contract,
- robust extraction from fenced/plain JSON (parse_ai_json),
- score normalization (clamp_score),
- threshold filtering based on user preference.

3.4 Integration of Functions Across System

- Anthropic API: JD metadata extraction, candidate scoring, onboarding enrichment.
- Supabase Storage: optional object storage for JD/resume files.
- Gmail SMTP: invite, decision, and reset-email communication.
- PostgreSQL: single source of truth for workflow state.

The API layer remains stable while service implementations can evolve independently.

---

4. System Configuration and Setup

4.1 Environment Setup

4.1.1 Prerequisites

- Python 3.11+ recommended
- PostgreSQL database reachable by DATABASE_URL
- Optional: Supabase project and Gmail app password

4.1.2 Install and Boot (Local)

```
cd /Users/mik/Desktop/code/QuickHire
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Initialize DB (first run):

```
python3 create_tables.py
python3 add_settings_columns.py
```

Run app:

```
python3 main.py
```

4.2 Configuration Variables

```
| Variable | Required | Purpose |
|---|---|---|
| SECRET_KEY | Yes | Flask session signing |
| DATABASE_URL | Yes | SQLAlchemy DB connection |
| ANTHROPIC_API_KEY | Recommended | AI analysis and scoring |
| GMAIL_ADDRESS | Optional | SMTP sender |
| GMAIL_APP_PASSWORD | Optional | SMTP auth |
| SUPABASE_URL | Optional | storage client init |
| SUPABASE_KEY | Optional | storage auth |
| ENABLE_SUPABASE_STORAGE | Optional | force storage on/off |
| SCREENING_BATCH_SIZE | Optional | candidates processed per screening request |
| RESUME_PARSE_MAX_PAGES | Optional | PDF extraction page limit |
| RESUME_PARSE_MAX_CHARS | Optional | extracted text cap |
| VERCEL | Platform-set | toggles serverless-safe defaults |
```

4.3 Deployment Process (Vercel Hobby)

4.3.1 Runtime Entry

vercel.json rewrites all routes to Flask entry:

```
{
  "version": 2,
  "rewrites": [
    { "source": "/(.*)", "destination": "/api/index.py" }
  ]
}
```

4.3.2 Hobby-Specific Optimizations

Implemented optimizations include:
- sequential per-file resume upload from UI,
- batch-based screening loops (processed, remaining, completed response shape),
- default storage disabled on Vercel unless explicitly enabled,
- bounded PDF extraction for upload path.

These reduce timeout probability under serverless execution budgets.

---

5. Testing and Validation

5.1 Current Testing Strategy

The current codebase is primarily validated through:
- route-level manual functional testing,
- browser flow verification (dashboard/job detail/settings flows),
- static checks:
- Python compile check (python3 -m py_compile ...),
- JS parse check (node --check static/js/*.js).

5.2 Validation Criteria by Function

```
| Function | Validation Criteria |
|---|---|
| upload_jd | PDF/text accepted, DB row created, invalid PDF rejected with JSON error |
| upload_resumes | valid PDF yields candidate records, failures return explicit reasons |
```

```
| start_screening | batch progress converges, statuses updated, ranked output returned |
| send_invites | ownership enforced, status updates, email dispatch result surfaced |
| final_decision | allowed status transition only, email sent result tracked |
| analytics_data | KPIs and chart payloads consistent with DB state |
```

5.3 Recommended Automated Test Expansion

Introduce pytest suites for:
- route auth and ownership gates,
- candidate status transition invariants,
- AI JSON parser edge cases,
- analytics aggregation correctness,
- upload error mapping (400/413/500 scenarios).

---

6. Challenges and Solutions

6.1 Serverless Timeouts in Resume/JD Flows

Challenge: processing multiple PDFs and multiple LLM calls in single request caused timeout risk.
Solution:
- sequential upload requests from frontend,
- batch scoring API with continuation semantics,
- extraction limits (max_pages, max_chars),
- reduced retry backoff in AI service.

6.2 Storage Latency and Optionality

Challenge: storage availability/latency should not block core screening workflow.
Solution:
- storage upload/deletion made best effort,
- storage defaults disabled on Vercel unless explicitly enabled,
- resume preview moved to DB text rendering path.

6.3 Invite Flow Inconsistency in Job Detail

Challenge: invite flow differed from dashboard behavior due modal/template and JS state handling defects.
Solution:
- modal correctly mounted in template modals block,
- scheduling link requirement aligned with backend contract,
- candidate ID state handling fixed for reliable status/badge updates.

6.4 User-Facing Error Ambiguity

Challenge: UI surfaced generic "network error" for non-200 responses.
Solution:
- parse non-OK responses and map status-specific messages,
- propagate backend JSON error details consistently.

---

7. Future Enhancements

7.1 Planned Improvements

1. Introduce async job queue for scoring (Celery/RQ/managed queue) to decouple LLM runtime from request

lifecycle.
2. Add OCR fallback for image-based PDFs.
3. Replace JSON-as-text skill fields with native JSONB columns for queryability.
4. Add multi-user RBAC and audit trail for enterprise workflows.
5. Expand analytics with cohort and time-to-hire metrics.
6. Add webhook integration (Slack/Teams/ATS connectors).

7.2 Scalability Considerations

- Move heavy analysis to background workers.
- Add idempotency keys for upload/screening requests.
- Introduce caching for repeated JD analysis and candidate report generation.
- Partition analytics computation into pre-aggregated tables for large tenants.
- Add structured observability (request IDs, latency metrics, error taxonomies).

---

8. Conclusion

QuickHire is implemented as a pragmatic, modular Flask platform optimized for operational hiring workflows and serverless constraints. The architecture balances AI flexibility with deterministic controls by combining strict prompt contracts, robust parsing, score normalization, ownership authorization, and workflow state gating.

The current implementation is production-capable for early-stage usage and already includes hardening for the most common operational risks (timeouts, inconsistent UI states, and error surfacing). The next technical frontier is asynchronous execution and deeper test automation to support larger candidate volumes and team concurrency.

Acknowledgments

This implementation builds on open-source ecosystems including Flask, SQLAlchemy, Anthropic SDK, pdfplumber, fpdf2, and Supabase.

Appendix A: API Endpoint Catalog

Auth and Session
- GET/POST /login
- GET/POST /register
- GET /logout

Password Reset
- GET/POST /forgot-password
- GET/POST /reset-password/<token>

Dashboard Views
- GET /dashboard/
- GET /dashboard/jobs
- GET /dashboard/candidates
- GET /dashboard/analytics
- GET /dashboard/settings

Job APIs
- POST /dashboard/upload-jd
- POST /dashboard/analyze-jd/<job_id>

- POST /dashboard/create-job
- GET /dashboard/job-detail/<job_id>
- GET /dashboard/jobs-filtered
- PATCH /dashboard/update-job-status/<job_id>
- DELETE /dashboard/delete-job/<job_id>

Candidate and Workflow APIs

- POST /dashboard/upload-resumes/<job_id>
- DELETE /dashboard/remove-resume/<candidate_id>
- POST /dashboard/start-screening/<job_id>
- POST /dashboard/screen-new-candidates/<job_id>
- GET /dashboard/results/<job_id>
- DELETE /dashboard/delete-candidate/<candidate_id>
- GET /dashboard/candidate-pdf/<candidate_id>
- GET /dashboard/resume-pdf/<candidate_id>
- POST /dashboard/send-invites
- POST /dashboard/final-decision
- POST /dashboard/send-custom-email
- GET /dashboard/generate-onboarding/<candidate_id>
- GET /dashboard/analytics-data
- GET /dashboard/analytics-export-csv

Settings and Data Management APIs

- POST /dashboard/settings/save
- POST /dashboard/settings/delete-data
- POST /dashboard/settings/close-account